

AWS EC2

Assignment Overview

This document provides complete step-by-step instructions for eight independent EC2 operations: launching instances with automated User Data scripts on both Amazon Linux 2 and Ubuntu, installing Tomcat on a Windows Server instance, creating an EBS snapshot, configuring passwordless SSH authentication, launching a Spot instance, enabling termination protection, and launching an instance using the AWS CLI.

The following instance/tool reference is used throughout this document:

Task	Instance	Tool Installed / Action
1	Amazon Linux 2 (Apache-UserData-EC2)	Apache (httpd) via User Data
2	Ubuntu 22.04 (Nginx-UserData-EC2)	Nginx via User Data
3	Windows Server 2022 (Tomcat-Windows-EC2)	Apache Tomcat
4	Instance from Task 1	EBS Snapshot
5	Instance from Task 2	Passwordless SSH authentication
6	Amazon Linux 2 (Spot-EC2)	Spot purchasing option
7	Instance from Task 2	Termination protection
8	Amazon Linux 2 (CLI-EC2)	AWS CLI launch

Task 1: Launch EC2 (Amazon Linux 2) with User Data to Install Apache

Outcome

An EC2 instance running Amazon Linux 2 is launched, and Apache (httpd) is automatically installed, started, and enabled during the first boot through a User Data script — with no manual SSH configuration required. The Apache test page is accessible at <http://<Public-IP>> immediately after the instance reaches the “Running” state.

Objective

To automate software installation at launch time using EC2 User Data, removing the need for any manual post-launch configuration and ensuring every instance boots into a consistent, ready-to-use state.

Prerequisites

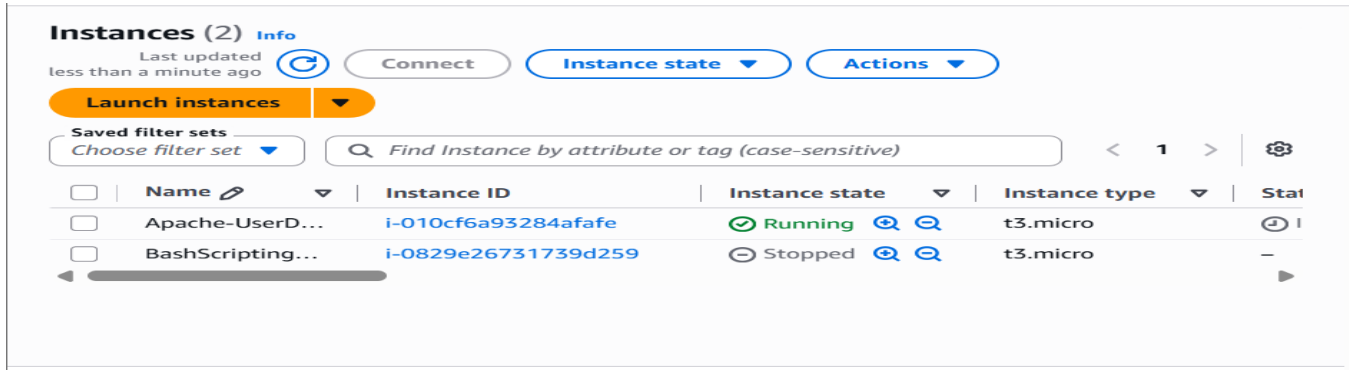
- AWS account with console access.
- IAM permissions to launch EC2 instances and manage security groups.
- An existing key pair for SSH access (used only for validation, not for installation).
- Region selected (e.g., ap-south-1 or us-east-1).

Step-by-Step Implementation

Step 1: Open the EC2 Console and Launch an Instance

Login to AWS Console → Search for “EC2” → Open EC2 Dashboard → Click “Launch instance”.

Screenshot Evidence:



Step 2: Configure the Instance

Parameter	Value
Name	Apache-UserData-EC2
Amazon Machine Image (AMI)	Amazon Linux AMI (HVM), SSD Volume Type
Instance type	t3.micro
Key pair	Select an existing key pair (e.g., multitool-key)

Step 3: Configure the Security Group

Type	Protocol	Port Range	Source
SSH	TCP	22	My IP
HTTP	TCP	80	Anywhere (0.0.0.0/0)

Screenshot Evidence:

▼ Security group rule 2 (TCP, 80, 0.0.0.0/0)
Remove

Type Info	Protocol Info	Port range Info
HTTP ▼	TCP	80
Source type Info	Source Info	Description - optional Info
Anywhere ▼	Add CIDR, prefix list or security group ID 0.0.0.0/0 ✕	e.g. SSH for admin desktop

▼ Security group rule 3 (TCP, 0)
Remove

Type Info	Protocol Info	Port range Info
Custom TCP ▼	TCP	0
Source type Info	Source Info	Description - optional Info
Custom ▼	Add CIDR, prefix list or security group ID	e.g. SSH for admin desktop

Add security group rule

Step 4: Add the User Data Script

Expand “Advanced details” at the bottom of the launch wizard → scroll to the “User data” field → paste the following script:

```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
echo "<h1>Apache Installed via User Data - Task 1</h1>" > /var/www/html/index.html
```

Screenshot Evidence :

```
#!/bin/bash
# Lanuch httpd by using UserData
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
echo "<h1>Apache Installed via User Data Task </h1>" >
/var/www/html/index.html
```

Step 5: Launch and Verify

Click “Launch instance” → wait until the instance state shows “Running” → copy the Public IPv4 address → open it directly in a browser (no SSH or manual installation needed):

`http://<Public-IP>`

Screenshot Evidence :



Apache Installed via User Data Task

Validation Steps

- Browser displays “Apache Installed via User Data - Task 1” without any manual installation step.
- Optional SSH check: `sudo systemctl status httpd` shows “active (running)”, confirming the script executed successfully at boot.

Explanation

User Data is a bootstrapping mechanism that EC2 passes to the instance's first-boot process (cloud-init on Linux), which executes the supplied script as root before the instance becomes available for use. Using it here to install Apache means the web server is guaranteed to be present and running the moment the instance launches, with zero manual steps. This is important because it demonstrates infrastructure-as-code style provisioning — the desired software state travels with the launch configuration itself rather than depending on someone remembering to SSH in and install it afterward.

Real-Time Use Case

Auto Scaling Groups and Launch Templates rely on User Data scripts to configure every new instance identically and automatically as the group scales up, which is essential for stateless, elastic web-tier architectures where instances are created and destroyed on demand without manual intervention.

Common Errors and Troubleshooting

Error	Cause	Resolution
Browser shows nothing / connection refused on port 80	Security group missing the inbound rule for port 80, or the User Data script failed	Add inbound TCP 80; SSH in and inspect <code>sudo cat /var/log/cloud-init-output.log</code> for script errors
User Data script appears to not run at all	Script missing the <code>#!/bin/bash</code> shebang line, or instance was stopped/started instead of freshly launched (User Data only runs on first boot by default)	Ensure the script starts with <code>#!/bin/bash</code> ; User Data executes only once on first boot unless configured otherwise
Page loads but shows the default Apache test page instead of the custom message	The <code>echo</code> command in the script did not execute due to a preceding command failing	Check <code>cloud-init-output.log</code> for the exact line that failed and correct it

Task 2: Launch EC2 (Ubuntu) with User Data to Install Nginx

Outcome

An EC2 instance running Ubuntu is launched, and Nginx is automatically installed, started, and enabled during first boot through a User Data script, with the Nginx welcome (or custom) page accessible at `http://<Public-IP>` immediately after launch.

Objective

To automate web server installation on a Debian-based distribution using User Data, and to demonstrate that the same automation approach applies across different Linux distributions with only the package-manager commands changing.

Prerequisites

- AWS account with console access.
- IAM permissions to launch EC2 instances and manage security groups.
- An existing key pair for SSH access (used only for validation).
- Region selected.

Step-by-Step Implementation

Step 1: Open the EC2 Console and Launch an Instance

Login to AWS Console → Search for “EC2” → Open EC2 Dashboard → Click “Launch instance”.

Step 2: Configure the Instance

Parameter	Value
Name	Nginx-UserData-EC2
Amazon Machine Image (AMI)	Ubuntu Server 26.04 LTS (HVM), SSD Volume Type
Instance type	t3.micro
Key pair	Select an existing key pair (e.g., multitool-key)

Screenshot Evidence :

your instance. If you don't see a suitable AMI below, use the search field or choose **Browse more AMIs**.

Q Search our full catalog including 1000s of application and OS images

Recents

Quick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

Canonical Ubuntu

Wi

Q

Browse more AMIs

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Ubuntu Server 26.04 LTS (HVM), SSD Volume Type

Free tier eligible

ami-0b6d9d3d33ba97d99 (64-bit (x86)) / ami-0bc7f2dbdcc6b5303 (64-bit (A

Virtualization: hvm ENA enabled: true Root device type: ebs

Ubuntu Server 26.04 L

Step 3: Configure the Security Group

Type	Protocol	Port Range	Source
SSH	TCP	22	My IP
HTTP	TCP	80	Anywhere (0.0.0.0/0)
HTTPS	TCP	443	Anywhere(0.0.0.0/0)

Screenshot Evidence :

Inbound Security Group Rules

▼ Security group rule 1 (TCP, 22, 103.203.172.230/32)

Remove

Type | Info

ssh ▼

Protocol | Info

TCP

Port range | Info

22

Source type | Info

My IP ▼

Name | Info

Q Add CIDR, prefix list or security group

103.203.172.230/32 ✕

Description - optional | Info

e.g. SSH for admin desktop

▼ Security group rule 2 (TCP, 443, 0.0.0.0/0)

Remove

Type | Info

HTTPS ▼

Protocol | Info

TCP

Port range | Info

443

Source type | Info

Anywhere ▼

Source | Info

Q Add CIDR, prefix list or security group

0.0.0.0/0 ✕

Description - optional | Info

e.g. SSH for admin desktop

▼ Security group rule 3 (TCP, 80, 0.0.0.0/0)

Remove

Type | Info

HTTP ▼

Protocol | Info

TCP

Port range | Info

80

Source type | Info

Anywhere ▼

Source | Info

Q Add CIDR, prefix list or security group

Description - optional | Info

e.g. SSH for admin desktop

Step 4: Add the User Data Script

Expand “Advanced details” → scroll to the “User data” field → paste the following script:

```
#!/bin/bash
apt update -y
apt install -y nginx
systemctl start nginx
systemctl enable nginx
echo "<h1>Nginx Installed via User Data - Task 2</h1>" > /var/www/html/index.html
```

Screenshot Evidence:

Choose file

```
#!/bin/bash
# Installing Nginx by UserData
apt update -y
apt install -y nginx
systemctl start nginx
systemctl enable nginx
echo "<h1>Nginx Installed Via User Data Task 2</h1>" > /var/www/html/index.html
```

Step 5: Launch and Verify

Click “Launch instance” → wait until the instance state shows “Running” → copy the Public IPv4 address → open it in a browser:

http://54.221.23.139:80

Screenshot Evidence:

← → ↻ ⚠ Not secure 54.221.23.139

Nginx Installed Via User Data Task 2

Validation Steps

- Browser displays “Nginx Installed via User Data - Task 2” without any manual installation step.
- Optional SSH check (username ubuntu): `sudo systemctl status nginx` shows “active (running)”.

Explanation

This task installs Nginx on Ubuntu using apt instead of yum, showing that the User Data automation concept is identical across distributions — only the package manager and default login user (ubuntu instead of ec2-user) change. This reinforces that boot-time automation is a portable pattern, not something tied to a single Linux flavor, which matters when an organization runs a mixed fleet of AMIs.

Real-Time Use Case

Many organizations standardize on Ubuntu for its package ecosystem and predictable release cadence. Teams commonly bake a startup script into the Launch Template so every Ubuntu web-tier instance automatically has Nginx installed and configured the same way, whether it was launched manually, by an Auto Scaling Group, or by a CI/CD pipeline.

Common Errors and Troubleshooting

Error	Cause	Resolution
“Permission denied (publickey)” when SSH-ing in	Wrong username used — Amazon Linux 2 uses ec2-user but Ubuntu uses ubuntu	Connect with <code>ssh -i multitool-key.pem ubuntu@<Public-IP></code>
Nginx page not reachable on port 80	Security group missing inbound rule for port 80, or apt install failed due to a locked package index	Add inbound TCP 80; check <code>/var/log/cloud-init-output.log</code> for an apt lock or network error and retry
User Data script did not run	Script missing the <code>#!/bin/bash</code> shebang, or instance was stopped/started rather than freshly launched	Confirm the shebang line is present; User Data runs only on first boot by default

Task 3: Launch a Windows Server EC2 and Install Apache Tomcat

Outcome

A Windows Server EC2 instance is launched, remote desktop (RDP) access is established, and Apache Tomcat is installed and running, with its welcome page accessible at `http://<Public-IP>:8080` from a browser.

Objective

To provision a Windows-based EC2 instance and host a Java Servlet/JSP container on it, demonstrating that Tomcat deployment is not limited to Linux and can run on Windows Server for organizations with a Windows-centric application stack.

Prerequisites

- AWS account with console access.
- IAM permissions to launch EC2 instances and manage security groups.
- A key pair to decrypt the Windows administrator password.
- An RDP client (Windows Remote Desktop Connection, or Microsoft Remote Desktop on Mac).

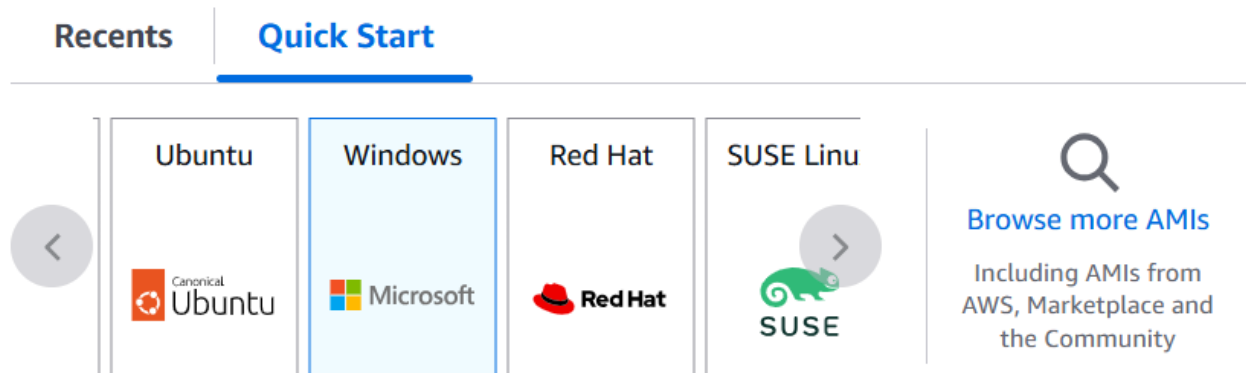
Step-by-Step Implementation

Step 1: Launch a Windows Server Instance

Login to AWS Console → EC2 Dashboard → Click “Launch instance” → configure:

Parameter	Value
Name	Tomcat-Windows-EC2
Amazon Machine Image (AMI)	Microsoft Windows Server 2025 Base
Instance type	T3.medium (Tomcat and the Windows GUI need more than 1 GiB of RAM)
Key pair	Select an existing key pair (e.g., multitool-key)

Screenshot Evidence :



Amazon Machine Image (AMI)

Microsoft Windows Server 2025 Base Free tier eligible
ami-09639480113b0df96 (64-bit (x86))
Virtualization: hvm ENA enabled: true Root device type: ebs

Description

Microsoft Windows 2025 Datacenter edition. [English]

Microsoft Windows Server 2025 Full Locale English AMI provided by Amazon

Step 2: Configure the Security Group

Type	Protocol	Port Range	Source
RDP	TCP	3389	My IP
Custom TCP (Tomcat)	TCP	8080	Anywhere (0.0.0.0/0)

Screenshot Evidence :

Inbound Security Group Rules

▼ Security group rule 1 (TCP, 3389, 103.203.172.230/32)

Remove

Type | Info

rdp

Protocol | Info

TCP

Port range | Info

3389

Source type | Info

My IP

Name | Info

Q Add CIDR, prefix list or security grou

Description - optional | Info

e.g. SSH for admin desktop

103.203.172.230/32 X

▼ Security group rule 2 (TCP, 0, 0.0.0.0/0)

Remove

Type | Info

Custom TCP

Protocol | Info

TCP

Port range | Info

0

Source type | Info

Anywhere

Source | Info

Q Add CIDR, prefix list or security grou

Description - optional | Info

e.g. SSH for admin desktop

0.0.0.0/0 X

Step 3: Retrieve the Administrator Password

After the instance reaches “Running”, select it → Click “Connect” → “RDP client” tab → “Get password” → upload the multitool-key.pem file → click “Decrypt password”.

Step 4: Connect via RDP

Open the Remote Desktop client on your computer → enter the instance's Public IPv4 address → sign in with username Administrator and the decrypted password.

Screenshot Evidence :



Step 5: Install Java (Required by Tomcat)

Inside the Windows desktop, open a browser and download the Java JDK (e.g., from the Amazon Corretto or Adoptium site) → run the installer with default options → verify from Command Prompt:

```
java -version
```

Step 6: Download and Extract Tomcat

Download the Windows zip distribution of Apache Tomcat from the official Apache Tomcat website → extract it to C:\Tomcat10 using File Explorer or the built-in “Extract All” option.

Step 7: Start Tomcat

Open Command Prompt as Administrator → navigate to the bin folder and run the startup script:

```
cd C:\Tomcat10\bin
startup.bat
```

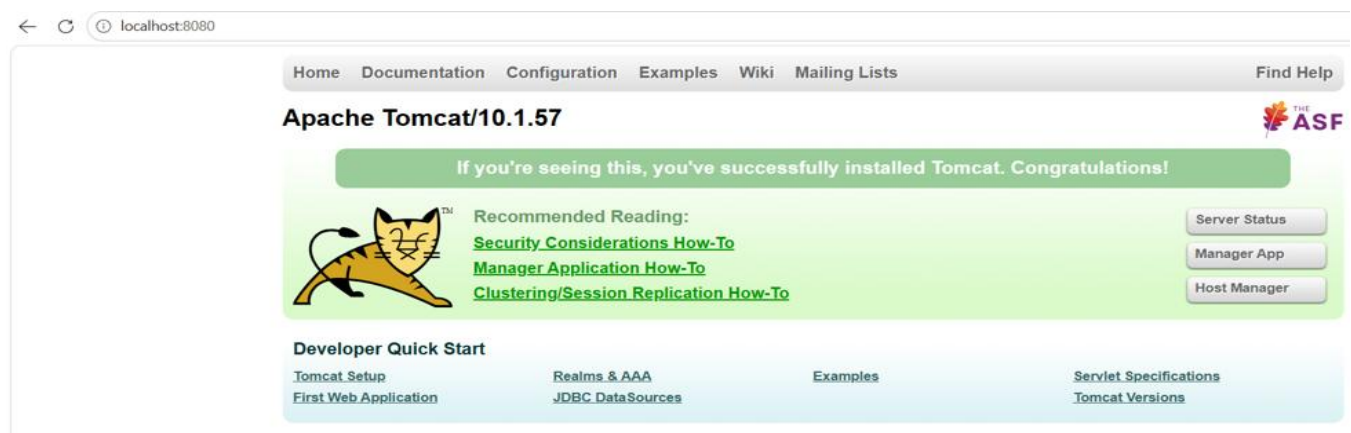
Step 8: Access Tomcat via Browser

From either the instance itself or from your local browser (since port 8080 is open in the security group):

```
http://<Public-IP>:8080
```

```
http://localhost:8080
```

Screenshot Evidence:



Validation Steps

- The Tomcat startup window remains open showing “Server startup in [xxxx] ms” with no errors.
- Browser displays the Apache Tomcat welcome page on port 8080.

Explanation

This task provisions a full Windows Server desktop environment in EC2 and manually installs Tomcat, since Windows does not use yum/apt package managers the way Linux does. Java is installed first because, as on Linux, Tomcat is a Java application and requires a JVM to run. Running Tomcat on Windows is important for organizations whose existing Java applications, licensing, or operational tooling are built around a Windows Server environment rather than Linux.

Real-Time Use Case

Some enterprises run legacy or regulated Java applications that were built and validated specifically for Windows Server, often alongside other Windows-only dependencies (such as certain drivers, COM components, or Active Directory integrations). Hosting Tomcat on Windows Server in EC2 lets these organizations lift-and-shift such applications into the cloud without re-platforming them to Linux.

Common Errors and Troubleshooting

Error	Cause	Resolution
RDP connection times out	Security group missing inbound rule for port 3389, or connecting from a different IP than allowed	Add/update the RDP inbound rule to allow the current IP
“The user name or password is incorrect” on RDP login	Wrong key pair used to decrypt the password, or password not yet available (instance still initializing)	Wait a few minutes after launch before requesting the password; confirm the correct .pem file is used
Tomcat page not reachable on port 8080	Security group missing inbound rule for 8080, Windows Firewall blocking the port, or startup.bat not run	Add inbound TCP 8080; allow the port through Windows Firewall; confirm startup.bat is running without errors

Task 4: Take a Snapshot of the Instance Created in Task 1

Outcome

An EBS snapshot of the root volume belonging to the Apache-UserData-EC2 instance (from Task 1) is created and reaches the “completed” state in the Snapshots console.

Objective

To create a point-in-time backup of the instance’s EBS volume, enabling recovery, volume duplication, or new-instance creation from a known good state.

Prerequisites

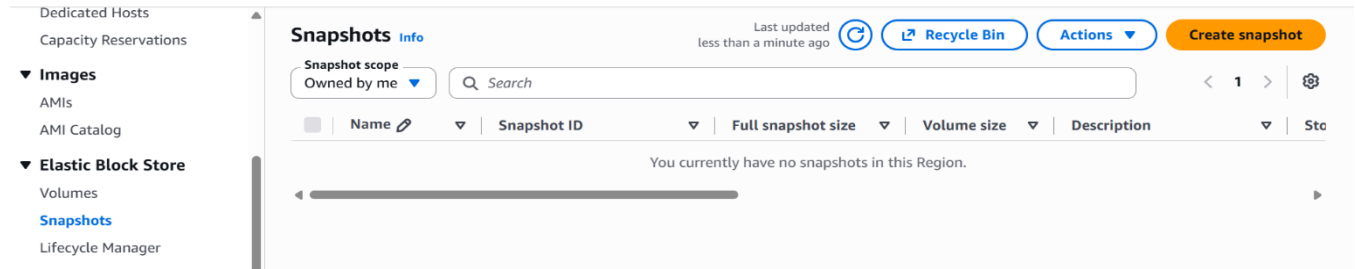
- Task 1 completed — Apache-UserData-EC2 instance running.
- IAM permissions to view volumes and create snapshots.

Step-by-Step Implementation

Step 1: Identify the Volume

EC2 Console → Elastic Block Store → Volumes → locate the volume whose “Attached instance” column shows Apache-UserData-EC2.

Screenshot Evidence:



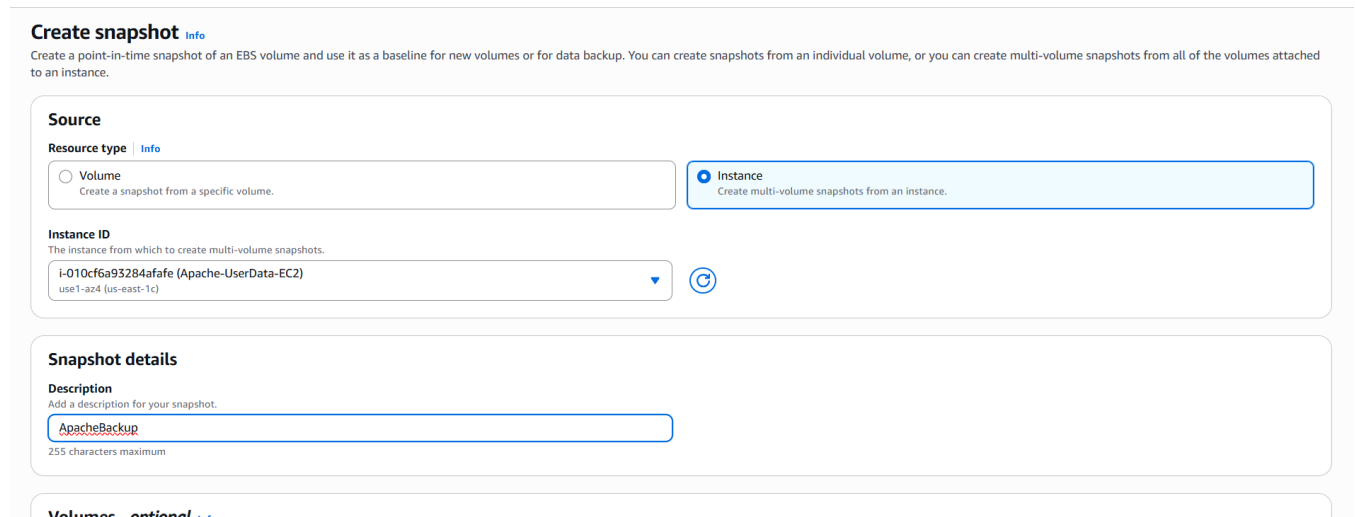
Step 2: Create the Snapshot

Select the volume → Actions → Create snapshot → enter:

Parameter	Value
Description	Apachebackup

Click “Create snapshot”.

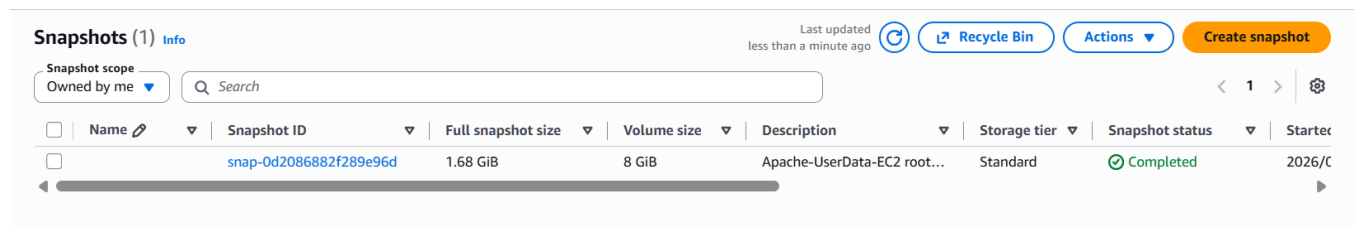
Screenshot Evidence :



Step 3: Monitor Snapshot Progress

Navigate to EC2 Console → Elastic Block Store → Snapshots → observe the “Status” column progress from “pending” to “completed”.

Screenshot Evidence :



Validation Steps

- The Snapshots list shows the new snapshot with Status “completed” and the correct Volume ID and description.

Explanation

An EBS snapshot is an incremental, point-in-time copy of a volume stored durably in Amazon S3 behind the scenes. Taking a snapshot of the Task 1 instance's volume captures the exact state of the operating system and the Apache installation at that moment, including everything the User Data script configured. This is important because it allows the instance to be restored, cloned into a new volume, or used as the basis for a custom AMI without repeating the original launch and configuration steps.

Real-Time Use Case

Organizations schedule regular EBS snapshots as part of their backup and disaster-recovery strategy, and take on-demand snapshots before risky changes (such as an OS patch or application upgrade) so they can quickly roll back by creating a new volume from the snapshot if something goes wrong.

Common Errors and Troubleshooting

Error	Cause	Resolution
Snapshot stuck in “pending” for a long time	Large volume size or high AWS backend load	Wait — snapshot duration depends on volume size and change rate since the last snapshot; no action needed
Cannot find the correct volume to snapshot	Multiple volumes exist and the attached-instance column was not checked	Filter or sort the Volumes list by “Attached instance” to locate the one attached to Apache-UserData-EC2
“UnauthorizedOperation” error when creating a snapshot	IAM user/role lacks ec2:CreateSnapshot permission	Attach or request a policy that grants ec2:CreateSnapshot on the relevant volume

Task 5: Assign Passwordless Authentication for the EC2 Created in Task 2

Outcome

The Ubuntu instance from Task 2 (Nginx-UserData-EC2) accepts SSH logins purely via an SSH key pair, with password authentication explicitly disabled at the server level, so no password is ever prompted or accepted.

Objective

To harden SSH access by enforcing key-based (passwordless) authentication and disabling password login entirely, which is a standard security best practice for any internet-facing Linux server.

Prerequisites

- Task 2 completed — Nginx-UserData-EC2 instance running.
- Existing SSH key pair used to launch the instance.
- sudo privileges on the instance.

Step-by-Step Implementation

Step 1: Connect to the Instance Using the Existing Key Pair

```
ssh -i multitool-key.pem ubuntu@54.224.90.153
```

Screenshot Evidence:

```
DELL@KHADEER MINGW64 ~/Downloads
$ ssh -i /c/Users/DELL/Downloads/multitool-key.pem ubuntu@54.224.90.152
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Jul 15 10:10:11 UTC 2026

System load:  0.0           Temperature:   -273.1 C
Usage of /:   39.8% of 6.61GB Processes:    120
Memory usage: 30%          Users logged in: 0
Swap usage:   0%           IPv4 address for ens5: 172.31.16.212

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

   https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

18 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ubuntu@ip-172-31-16-212:~$ |
```

Step 2: Generate a Local SSH Key Pair (If One Does Not Already Exist)

On your local machine:

```
cat ~/.ssh/id_ed25519.pub
```

Press Enter to accept the default file location and an empty passphrase for fully passwordless login.

Screenshot Evidence:

```
DELL@KHADEER MINGW64 ~/Downloads
$ ls -la ~/.ssh
total 42
drwxr-xr-x 1 DELL 197121  0 Jul 14 18:41 ./
drwxr-xr-x 1 DELL 197121  0 Jul 14 20:26 ../
-rw-r--r-- 1 DELL 197121 399 Jul  7 15:47 id_ed25519
-rw-r--r-- 1 DELL 197121  94 Jul  7 15:47 id_ed25519.pub
-rw-r--r-- 1 DELL 197121 3389 Jul  3 12:11 id_rsa
-rw-r--r-- 1 DELL 197121  752 Jul  3 12:11 id_rsa.pub
-rw-r--r-- 1 DELL 197121 3900 Jul 14 18:41 known_hosts
-rw-r--r-- 1 DELL 197121 3726 Jul 14 18:41 known_hosts.old

DELL@KHADEER MINGW64 ~/Downloads
$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIGY8af0Q77CUDTyhVjE0RYa2jfcSEZZRTRfjZSVxj+j
DELL@KHADEER
```

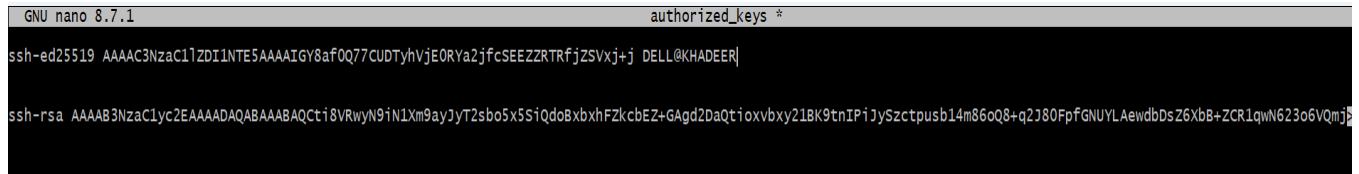
Step 3: Copy the Public Key to the Instance

From your local machine:

```
cd ~/.ssh  
nano authorized
```

Or manually append the public key content to ~/.ssh/authorized_keys on the instance.

Screenshot Evidence:



A screenshot of a terminal window showing the nano text editor editing the file ~/.ssh/authorized_keys. The editor title bar shows 'GNU nano 8.7.1' and 'authorized_keys *'. The file content contains two SSH keys: an ssh-ed25519 key and an ssh-rsa key, both preceded by their respective permissions (chmod 600).

Step 4: Disable Password Authentication on the Server

On the instance, edit the SSH daemon configuration:

```
sudo vi /etc/ssh/sshd_config
```

Ensure the following lines are set:

```
PasswordAuthentication no  
PubkeyAuthentication yes
```

Save and exit, then restart the SSH service:

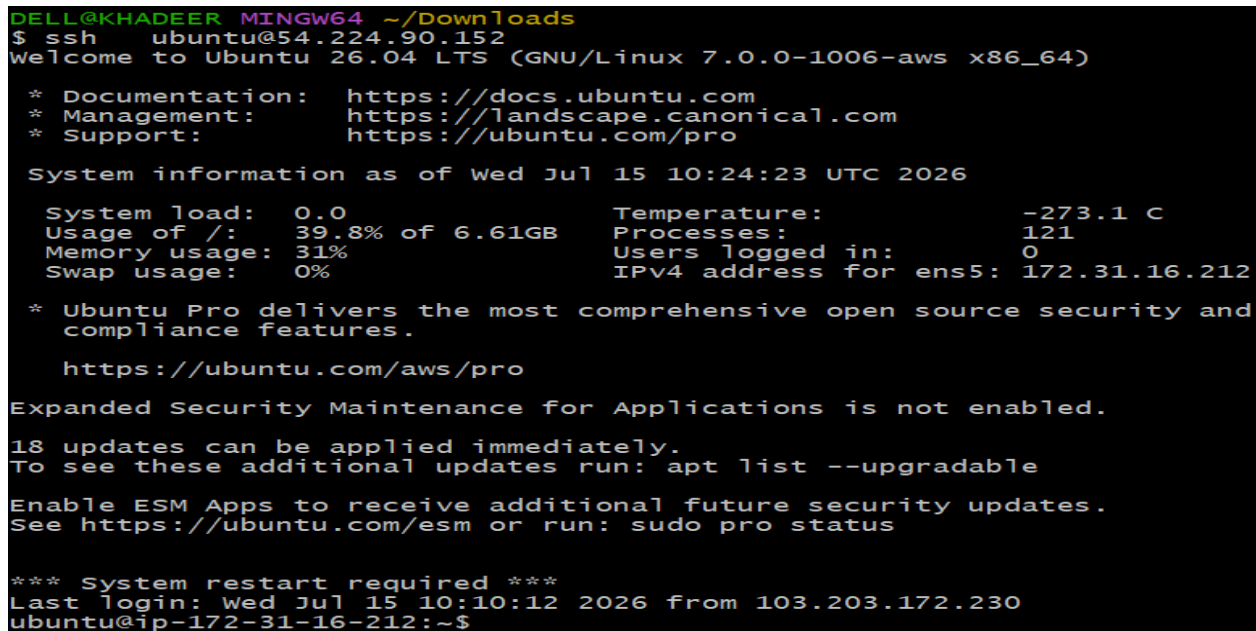
```
sudo systemctl restart ssh
```

Step 5: Verify Passwordless Login

From your local machine, connect using only the key — no password prompt should appear:

```
ssh : ubuntu@54.224.90.152
```

Screenshot Evidence:



A screenshot of a terminal window showing an SSH login session. The prompt is DELL@KHADEER MINGW64 ~/Downloads. The user runs \$ ssh ubuntu@54.224.90.152. The terminal displays the Ubuntu 26.04 LTS welcome message, system information (Wed Jul 15 10:24:23 UTC 2026), and a list of 18 updates that can be applied immediately. The user is prompted to run apt list --upgradable to see these updates. The terminal also shows the Ubuntu Pro status and a message indicating that a system restart is required.

Validation Steps

- SSH login succeeds immediately using the private key with no password prompt at any point.

- Attempting `ssh ubuntu@<Public-IP>` without a valid key is rejected outright, confirming password authentication is disabled.

Explanation

Passwordless SSH authentication relies on asymmetric cryptography: the private key stays on the client and the corresponding public key is stored in the server's `authorized_keys` file, so the server can verify identity without ever transmitting or storing a password. Disabling `PasswordAuthentication` in `sshd_config` closes off brute-force password-guessing attacks entirely, since the server will not accept a password even if one is supplied. This is important because it significantly reduces the attack surface of any internet-facing instance.

Real-Time Use Case

Production environments and configuration-management tools such as Ansible rely on passwordless SSH to connect to fleets of servers non-interactively for automated deployments, and security baselines in most organizations mandate disabling password-based SSH login on all cloud instances.

Common Errors and Troubleshooting

Error	Cause	Resolution
"Permission denied (publickey)" after disabling password auth	Public key was not correctly copied to <code>authorized_keys</code> , or file permissions are too open	Verify <code>~/.ssh/authorized_keys</code> contains the correct public key; run <code>chmod 700 ~/.ssh</code> and <code>chmod 600 ~/.ssh/authorized_keys</code>
Locked out of the instance entirely	<code>sshd_config</code> was misconfigured before confirming key-based access worked	Use EC2 Instance Connect or the AWS Systems Manager Session Manager (if configured) to fix <code>sshd_config</code> , or restore from the Task 4 snapshot
<code>ssh-copy-id</code> command not found	Utility not installed on the local machine (common on some Windows setups)	Manually copy the contents of <code>id_rsa.pub</code> into the instance's <code>~/.ssh/authorized_keys</code> file instead

Task 6: Launch an EC2 Instance Using the Spot Purchasing Option

Outcome

An EC2 instance is launched using the Spot purchasing option, reaches the "Running" state, and shows "spot" as its lifecycle in the console — running at a lower cost than an equivalent On-Demand instance.

Objective

To provision compute capacity using spare AWS capacity at a significant discount, suitable for fault-tolerant or flexible workloads that can tolerate interruption.

Prerequisites

- AWS account with console access.
- IAM permissions to launch EC2 instances and request Spot capacity.
- Understanding that Spot instances can be reclaimed by AWS with a two-minute interruption notice.

Step-by-Step Implementation

Step 1: Open the EC2 Console and Launch an Instance

Login to AWS Console → EC2 Dashboard → Click "Launch instance".

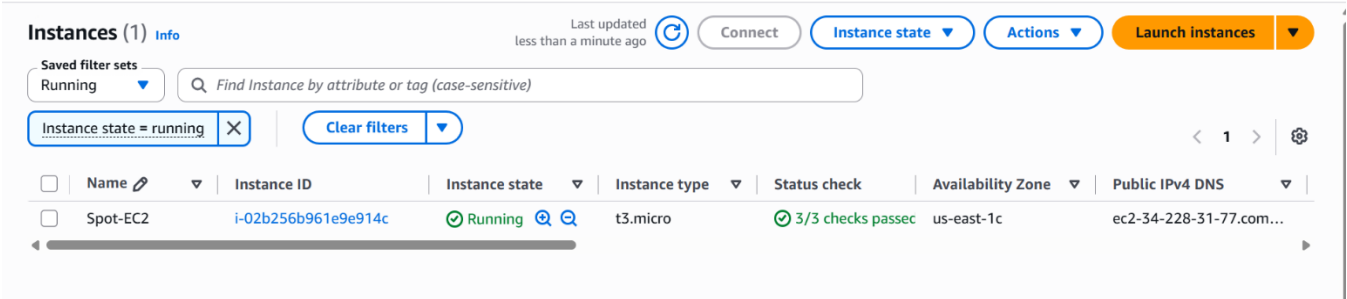
Step 2: Configure the Basic Instance Settings

Parameter	Value
Name	Spot-EC2
Amazon Machine Image (AMI)	Amazon Linux AMI (HVM), SSD Volume Type
Instance type	t3.micro

Step 4: Launch and Verify

Click “Launch instance” → wait for the instance state to show “Running”.

Screenshot Evidence:



Validation Steps

- The Instances list shows Spot-EC2 in the “Running” state.
- EC2 Console → Spot Requests shows a corresponding request with status “fulfilled”.

Explanation

Spot instances let AWS customers bid for unused EC2 capacity at up to a 90% discount compared to On-Demand pricing, in exchange for the possibility that AWS may reclaim the instance with a two-minute warning when that capacity is needed elsewhere. Choosing the Spot purchasing option here demonstrates how the same AMI and instance type can be launched more cheaply for workloads that tolerate interruption, which is an important cost-optimization lever in real AWS billing.

Real-Time Use Case

Organizations use Spot instances for batch processing, big-data analytics, CI/CD build agents, and stateless web-tier capacity in Auto Scaling Groups, where an occasional interruption is acceptable in exchange for substantially lower compute costs.

Common Errors and Troubleshooting

Error	Cause	Resolution
“There is no Spot capacity available”	Insufficient Spot capacity for the chosen instance type/Availability Zone at that time	Retry, choose a different instance type, or allow multiple Availability Zones
Instance terminated unexpectedly	AWS reclaimed the Spot capacity due to an interruption	Expected Spot behavior — design workloads to checkpoint progress or use a Spot Fleet/Auto Scaling Group for resilience
Spot request stuck in “pending-evaluation”	Requested price or configuration cannot currently be fulfilled	Wait, adjust the maximum price, or remove the price cap to use the default (up to On-Demand price)

Task 7: Enable Termination Protection on the EC2 Created in Task 2

Outcome

Termination protection is enabled on the Nginx-UserData-EC2 instance from Task 2, so that attempting to terminate it from the console or CLI fails with an explicit error until protection is disabled.

Objective

To prevent accidental termination of a critical instance, adding a safeguard against human error in the console or automated scripts.

Prerequisites

- Task 2 completed — Nginx-UserData-EC2 instance running.
- IAM permissions to modify instance attributes.

Step-by-Step Implementation

Step 1: Open the Instance Settings

EC2 Console → Instances → select Nginx-UserData-EC2 → Actions → Instance settings → Change termination protection.

Screenshot Evidence:

Change termination (deletion) protection

To prevent your instance from being accidentally deleted, you can enable termination protection for the instance. [Learn more](#)

Instance ID
i-01506b5d14d0b8618 (Nginx-UserData-Ec2)

Termination protection
☐ Enable

[Cancel](#) [Save](#)

Step 2: Enable Termination Protection

In the dialog box, check “Enable” → click “Save”.

Screenshot Evidence:

Terminate (delete) instance

⚠ On an EBS-backed instance, the default action is for the root EBS volume to be deleted when the instance is terminated. Storage on any local drives will be lost.

Are you sure you want to terminate these instances?

Instance ID	Termination protection
i-010cf6a93284afafe (Apache-UserData-EC2)	⚠ Enabled

To confirm that you want to delete the instances, choose the terminate button below. Instances with termination protection enabled will not be terminated. Terminating the instance cannot be undone.

Skip OS shutdown
This option skips the graceful OS shutdown process. Use only when your instance must be stopped immediately, such as during an emergency or failover.

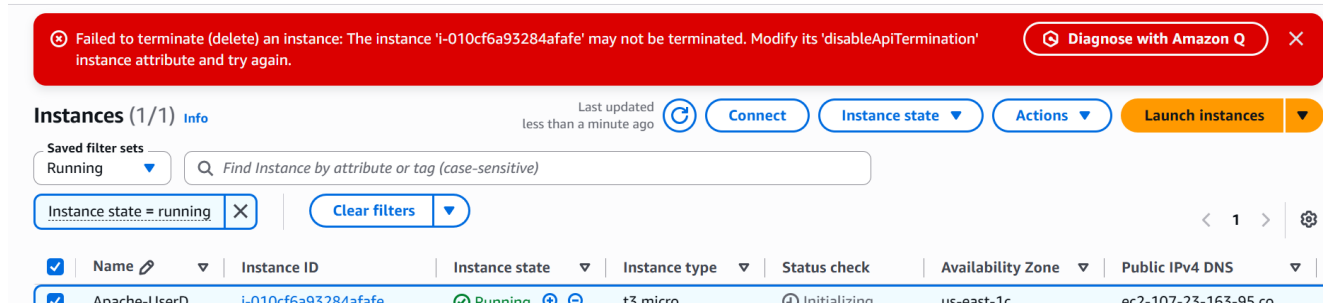
☐ Skip OS shutdown

[Cancel](#) [Terminate \(delete\)](#)

Step 3: Attempt to Terminate the Instance (Verification Attempt)

Select the instance → Actions → Instance state → Terminate instance → confirm → observe the resulting error.

Screenshot Evidence:



Validation Steps

- The termination attempt fails with a message similar to: “The instance ... may not be terminated. Modify its ‘disableApiTermination’ instance attribute and try again.”
- The instance remains in the “Running” state after the failed termination attempt.

Explanation

Termination protection sets the `disableApiTermination` attribute on the instance to true, which causes the EC2 API to reject any termination request — whether initiated from the console, the CLI, or the API — until the attribute is explicitly turned off. This is important because it guards against a single accidental click or a scripting mistake permanently destroying an instance and its data (though it does not protect against OS-level shutdown or stopping the instance).

Real-Time Use Case

Production databases, license servers, and other stateful or hard-to-rebuild instances are commonly protected this way so that a mistaken termination command from an engineer or a faulty automation script cannot destroy them without a deliberate, separate step to disable protection first.

Common Errors and Troubleshooting

Error	Cause	Resolution
Instance terminates despite expecting protection	Termination protection was not actually enabled, or was disabled again before the attempt	Re-check Instance settings → Change termination protection to confirm “Enable” is checked and saved
Cannot find “Change termination protection” option	Option is under Actions → Instance settings, easy to overlook among other menu items	Navigate precisely: Actions → Instance settings → Change termination protection
Need to actually terminate the instance later	Termination protection intentionally blocks this	Repeat the same steps and uncheck “Enable” to disable protection before terminating

Task 8: Launch an EC2 Instance Using the AWS CLI

Outcome

A new EC2 instance is launched entirely from the command line using the AWS CLI, without using the AWS Management Console, and is confirmed running via a CLI describe command.

Objective

To perform EC2 provisioning programmatically, which is the foundation for scripting, automation, and Infrastructure-as-Code workflows.

Prerequisites

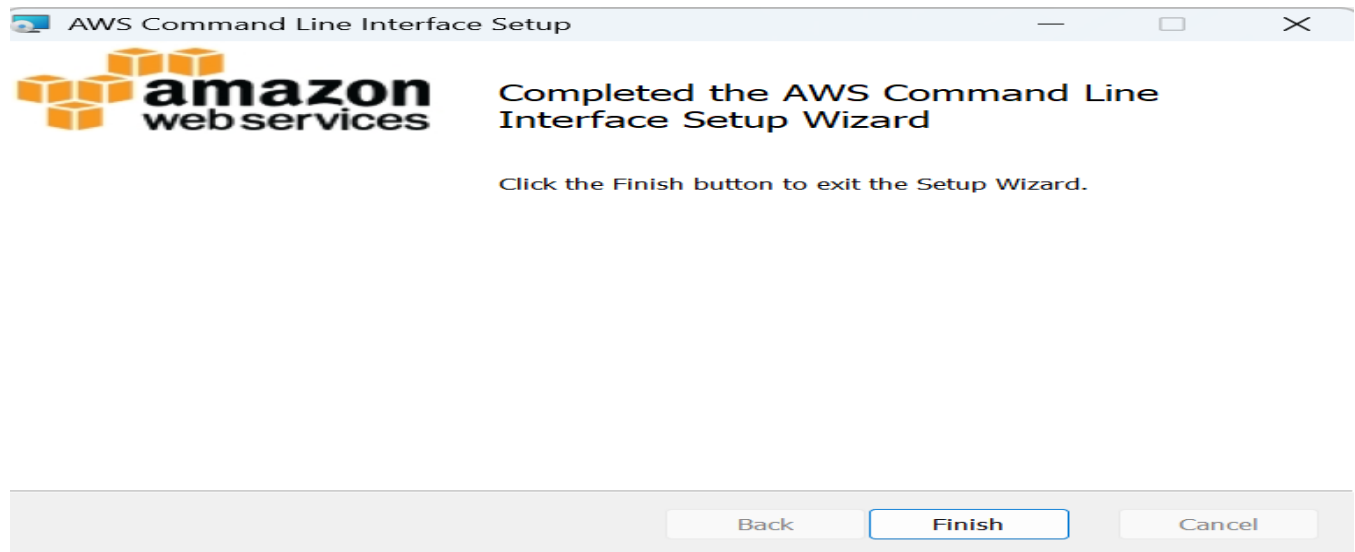
- AWS CLI installed on the local machine or a management host.
- An IAM user or role with an access key/secret key and EC2 permissions.
- An existing key pair, security group ID, and subnet ID to reference in the command.

Step-by-Step Implementation

Step 1: Install the AWS CLI

Download On Windows:

Screenshot Evidence:



Step 2: Configure the CLI with Credentials

```
aws --version  
aws configure
```

Enter the AWS Access Key ID, Secret Access Key, default region (e.g., ap-south-1), and output format (e.g., json) when prompted.

Screenshot Evidence:

```
DELL@KHADEER MINGW64 ~  
$ aws --version  
aws-cli/1.45.48 Python/3.14.4 windows/11 botocore/1.43.48  
  
DELL@KHADEER MINGW64 ~  
$ aws configure  
AWS Access Key ID [None]:  
AWS Secret Access Key [None]:  
Default region name [None]:  
Default output format [None]:  
  
DELL@KHADEER MINGW64 ~  
$ ^C  
  
DELL@KHADEER MINGW64 ~  
$ aws configure  
AWS Access Key ID [None]: AKIAYHJC5RPA5T4DH5WR  
AWS Secret Access Key [None]: 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  
Default region name [None]: ap-south-2  
Default output format [None]:
```

Step 3: Launch the Instance

```
aws ec2 run-instances \  
  --image-id ami-0abcdef1234567890 \  
  --instance-type t3.micro \  
  --key-name multitool-key \  
  --region ap-south-2
```

Screenshot Evidence:

```
DELL@KHADEER MINGW64 ~  
$ aws ec2 run-instances \  
> --image-id ami-0960ac155146d21bd \  
> --instance-type t3.micro \  
> --key-name "Challenge" \  
> --region ap-south-2
```

Step 4: Verify the Instance Is Running

```
aws ec2 describe-instances
```

Screenshot Evidence:

```
{  
  "ReservationId": "r-09efd4a9674737703",  
  "OwnerId": "565398309825",  
  "Groups": [],  
  "Instances": [  
    {  
      "Architecture": "x86_64",  
      "BlockDeviceMappings": [],  
      "ClientToken": "bcd91558-b5f8-478a-8fd4-ad5840ef2ad7",  
      "EbsOptimized": false,  
      "EnaSupport": true,  
      "Hypervisor": "xen",  
      "NetworkInterfaces": [  
        {  
          "Attachment": {  
            "AttachTime": "2026-07-15T11:26:09.000Z",  
            "AttachmentId": "eni-attach-05bd49beac356calc",  
            "DeleteOnTermination": true,  
            "DeviceIndex": 0,  
            "Status": "attaching",  
            "NetworkCardIndex": 0  
          },  
          "Description": "",  
          "Groups": [  
            {  
              "GroupId": "sg-0f21ce39a99c9abf5",  
              "GroupName": "default"  
            }  
          ],  
          "Ipv6Addresses": [],  
          "MacAddress": "06:1a:25:e6:e8:ad",  
          "NetworkInterfaceId": "eni-00fca9902116c7607",  
          "OwnerId": "565398309825",  
          "PrivateDnsName": "ip-172-31-15-39.ap-south-2.compute.internal",  
          "PrivateIpAddress": "172.31.15.39",  
          "PrivateIpAddresses": [  
            {  
              "Primary": true,  
              "PrivateDnsName": "ip-172-31-15-39.ap-south-2.compute.internal",  
              "PrivateIpAddress": "172.31.15.39"  
            }  
          ]  
        }  
      ]  
    }  
  ]  
}
```

Validation Steps

- The run-instances command returns a JSON response containing an Instance ID with an initial state of “pending”.
- The describe-instances query returns "running" once the instance has fully started, and CLI-EC2 also appears in the EC2 console.

Explanation

The AWS CLI wraps the same underlying EC2 API calls that the Management Console uses internally, so launching an instance with `run-instances` produces an identical result to using the console wizard — only expressed as a single reproducible command instead of a sequence of clicks. This is important because CLI commands can be scripted, version-controlled, and reused, forming the basis for automation and Infrastructure-as-Code tools such as Terraform or CloudFormation, which ultimately call these same APIs.

Real-Time Use Case

DevOps engineers use the AWS CLI inside shell scripts, CI/CD pipelines, and cron jobs to provision or tear down infrastructure automatically — for example, spinning up temporary test environments as part of a Jenkins pipeline, or scaling infrastructure based on a scheduled or event-driven trigger without any human touching the console.

Common Errors and Troubleshooting

Error	Cause	Resolution
“Unable to locate credentials”	aws configure was not run, or credentials file is missing/misnamed	Run <code>aws configure</code> and supply valid Access Key ID and Secret Access Key
“InvalidAMIID.NotFound”	The AMI ID used does not exist in the selected region (AMI IDs are region-specific)	Look up the correct AMI ID for the target region using <code>aws ec2 describe-images</code>
“UnauthorizedOperation” on <code>run-instances</code>	IAM user/role lacks <code>ec2:RunInstances</code> or related permissions	Attach a policy granting the required EC2 permissions to the IAM user or role